

Dynamic Programming

Backgrounds

- **Explores the space of all *possible solutions***
 - Decompose a problem into a series of subproblems
 - Build up correct solutions to larger and larger subproblems

Backgrounds

- The term *Dynamic Programming* comes from Control Theory
- *Programming* refers to the use of tables (arrays) to construct a solution.
- Used extensively in "Operation Research" taught in the Math dept.

Key Idea

- Dynamic programming usually *reduces time* by using *more space*
- Solve the problem by solving subproblems of increasing size, while saving each solution for a subproblem in a table
- Use the table to find the solution to larger problems.
- Time is saved since each subproblem is solved only once

Steps to Design a Dynamic Programming Algorithm

- Establish a recursive property giving the solution to an instance of the problem
- Solve an instance of the problem in a **bottom-up** fashion by solving smaller instances first
 - Recall that divide & conquer is **top-down**
 - Divide and conquer blindly divides a problem into smaller instances
 - A divide & conquer algorithm, e.g., the recursive n^{th} Fibonacci number algorithm, may solve the same instance more than once
→ Slow

Fibonacci's Series

Fibonacci's Series:

- **Definition:** $S_0 = 0,$

$$S_1 = 1,$$

$$S_n = S_{n-1} + S_{n-2} \quad n > 1$$

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

- Applying the recursive definition we get:

fib (*n*)

1. **if** *n* < 2

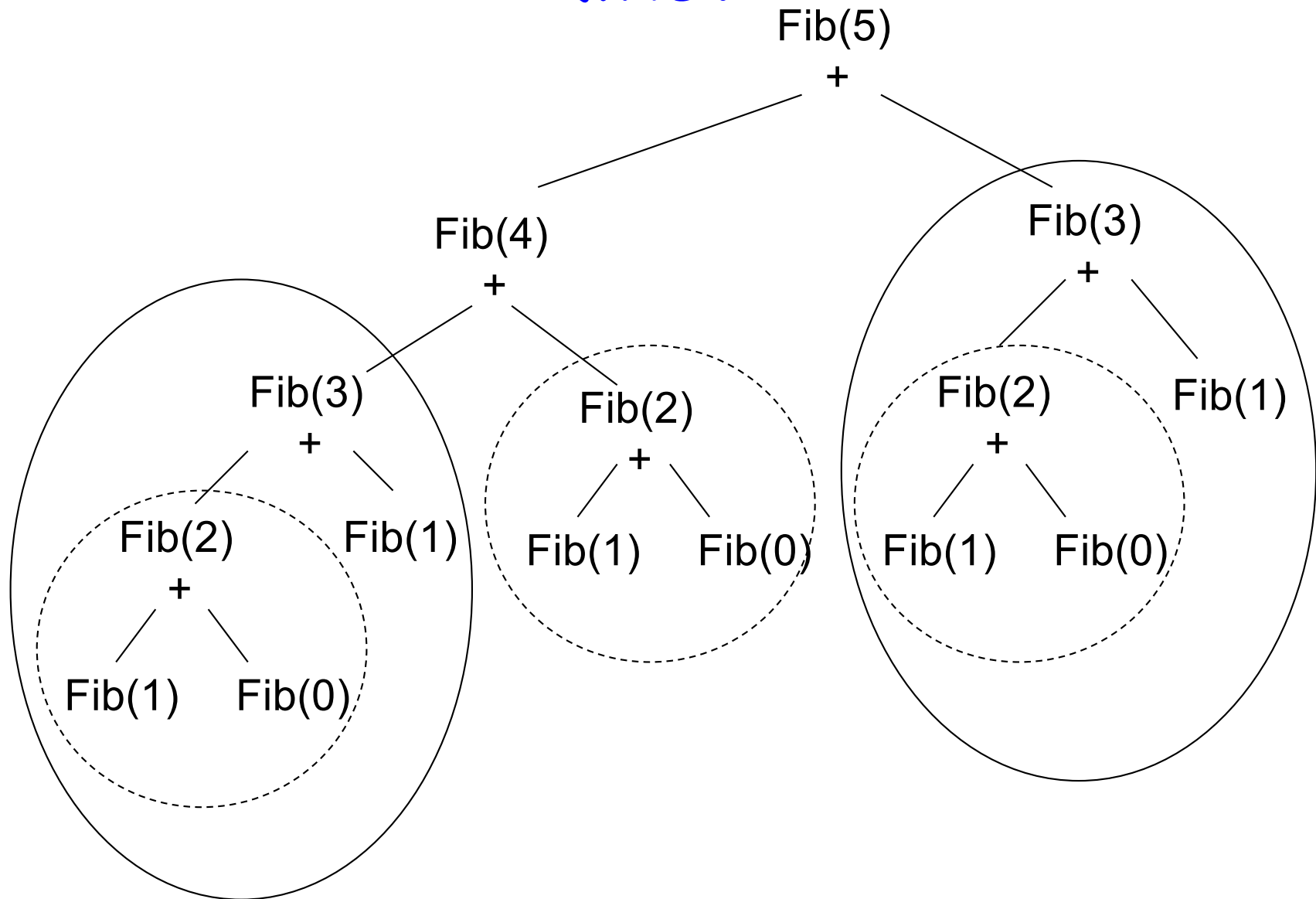
2. **return** *n*

3. **else return** (*fib* (*n* - 1) + *fib*(*n* - 2))

Analysis using Substitution of Recursive Fibonacci

- Let $T(n)$ be the number of additions done by $\text{fib}(n)$
- $T(n) = T(n-1) + T(n-2) + 1$ for $n \geq 2$
 $T(2) = 1$
- $T(n) \in O(2^n)$, $T(n) \in \Omega(2^{n/2})$

What does the Recursion Tree look like?



Dynamic Programming Solution for Fibonacci

- Build a table with the first n Fibonacci numbers.

fib(n)

1. $A[0] = 0$

2. $A[1] = 1$

3. for $i \leftarrow 2$ to n

4. $A[i] = A[i-1] + A[i-2]$

5. return A

Is there a recurrence equation?

How many additions are needed?

What is the space requirements?

If we only need the n^{th} number can we save space?

Binomial Coefficient

Binomial Coefficient

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} \text{ for } 0 < k < n$$

$$\binom{n}{k} = \begin{cases} \binom{n-1}{k-1} + \binom{n-1}{k} & 0 < k < n \\ 1 & k = 0 \text{ or } k = n \end{cases}$$

Recursive algorithm

`binomialCoef( $n$ ,  $k$ )`

1. **if** $k = 0$ or $k = n$

2. **return** 1

3. **else**

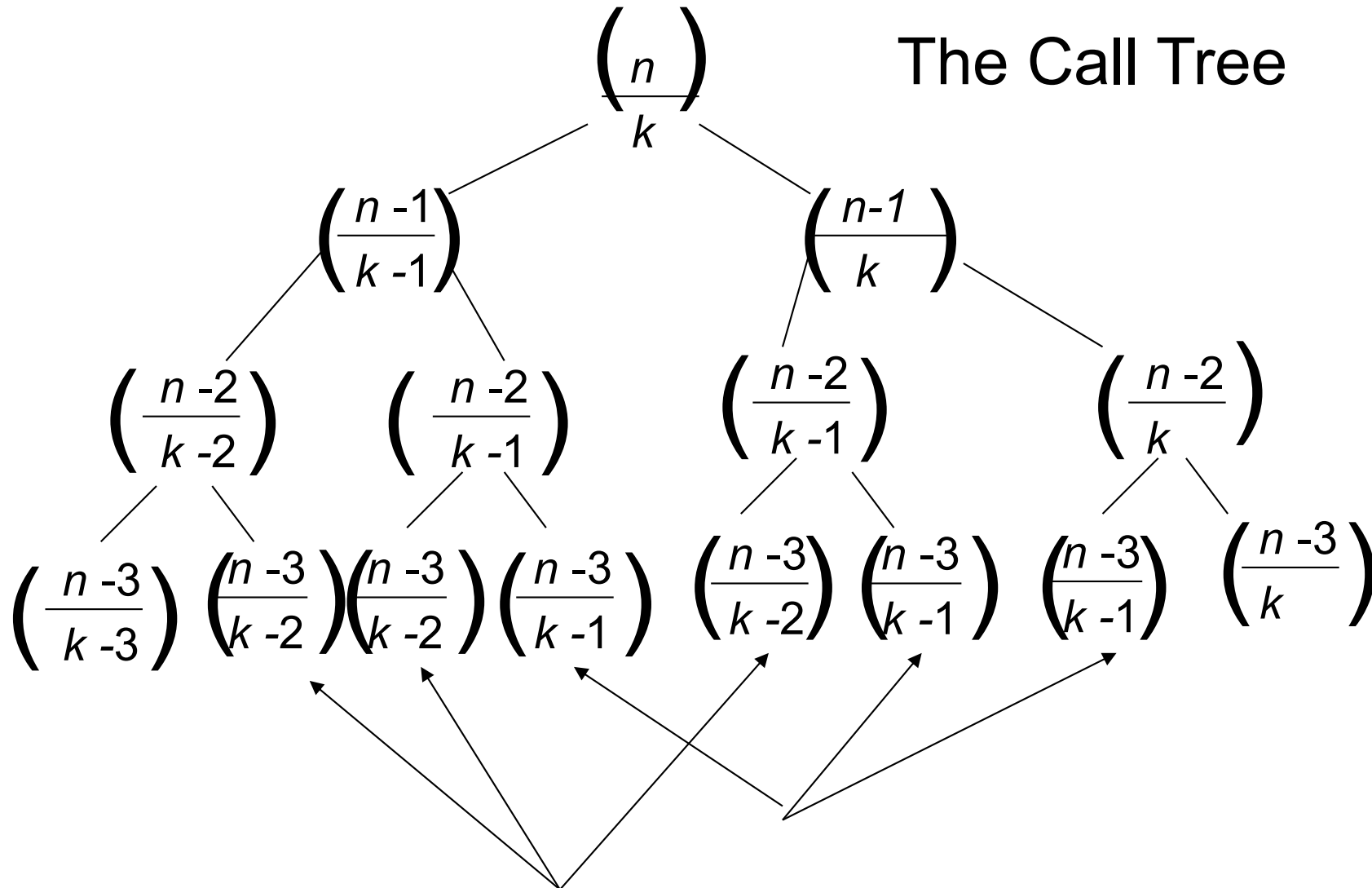
return (`binomialCoef( $n-1$ ,  $k-1$ )` + `binomialCoef( $n-1$ ,  $k$ )`)

$\text{binomialCoef}(n, k)$

1. **if** $k = 0$ or $k = n$

2. **then return** 1

3. **else return** $\text{binomialCoef}(n-1, k-1) + \text{binomialCoef}(n-1, k)$



Dynamic Programming Solution

- Use a matrix B of $n+1$ rows, $k+1$ columns where

$$B[n, k] = \binom{n}{k}$$

- *Establish* a recursive property. Rewrite in terms of matrix B :
$$B[i, j] = \begin{cases} B[i-1, j-1] + B[i-1, j] & \text{if } 0 < j < i \\ 1 & \text{if } j = 0 \text{ or } j = i \end{cases}$$
- Solve all “smaller instances of the problem” in a *bottom-up* fashion by computing the rows in B in sequence starting with the first row.

The B Matrix

	0	1	2	3	4 ...	j	k
0	1						
1	1	1					
2	1	2	1				
3	1	3	3	1			
4	1	4	6	4	1		
				$B[i-1, j-1]$		$B[i-1, j]$	
i						$B[i, j]$	
n							

Compute $B[4,2] = \binom{4}{2}$

- Row 0: $B[0,0] = 1$
- Row 1: $B[1,0] = 1$
 $B[1,1] = 1$
- Row 2: $B[2,0] = 1$
 $B[2,1] = B[1,0] + B[1,1] = 2$
 $B[2,2] = 1$
- Row 3: $B[3,0] = 1$
 $B[3,1] = B[2,0] + B[2,1] = 3$
 $B[3,2] = B[2,1] + B[2,2] = 3$
- Row 4: $B[4,0] = 1$
 $B[4,1] = B[3,0] + B[3,1] = 4$
 $B[4,2] = B[3,1] + B[3,2] = 6$

Dynamic Program

- $\text{bin}(n, k)$
 1. **for** $i = 0$ **to** n // every row
 2. **for** $j = 0$ **to** $\text{minimum}(i, k)$
 3. **if** $j == 0$ or $j == i$ // column 0 or diagonal
 4. $B[i, j] = 1$
 5. **else**
 6. $B[i, j] = B[i-1, j-1] + B[i-1, j]$
 7. **return** $B[n, k]$
- What is the run time?
- How much space does it take?
- If we only need the last value, can we save space?

Dynamic programming

- All values in column 0 are 1
- All values in the $k+1$ diagonal cells are 1
- $j \neq i$ and $0 < j \leq \min\{i, k\}$ ensures we only compute $B[i, j]$ for $j < i$ and only first $k+1$ columns
- **Elements above diagonal ($B[i, j]$ for $j > i$) are not computed since $\binom{i}{j}$ is undefined for $j > i$**

Number of iterations

i: 0, 1, 2, ..., k, k+1, ..., n

#iterations: 1, 2, 3, ...k+1, k+1, ..., k+1

$$\begin{aligned} \sum_{i=0}^n \sum_{j=0}^{\min(i,k)} 1 &= \sum_{i=0}^k \sum_{j=0}^i 1 + \sum_{i=k+1}^n \sum_{j=0}^k 1 = \\ \sum_{i=0}^k (i+1) + \sum_{i=k+1}^n (k+1) &= \\ \frac{(k+2)(k+1)}{2} + (n-k)(k+1) &= \\ \frac{(2n-k+2)(k+1)}{2} &= \theta(nk) \end{aligned}$$

Dynamic Programming to solve an Optimization Problem

- Used for problems in which an **optimal solution for the original problem can be found from optimal solutions to subproblems** of the original problem
- A recursive algorithm based on the divide and conquer approach can solve the problem. But the recursive algorithm may compute the solution to the same subproblem more than once and, therefore, it is slow.
 - Fibonacci term and binomial coefficient have such inefficient recursive algorithms
 - $\text{fibonacci}(n) = \text{fibonacci}(n-1) + \text{fibonacci}(n-2)$
 - More details on Fibonacci term and binomial coefficient later
- To reduce time, compute the optimal solution of a subproblem **only once** and save its value. Reuse the saved value whenever the same subproblem needs to be solved.

Principle of Optimality – Optimal Substructure

- Principle of optimality
 - Given an optimal sequence of decisions or choices, each subsequence must also be optimal.
- The principle of optimality applies to a *problem* (not an algorithm)
 - A large number of optimization problems satisfy this principle. Examples follow.

Principle of optimality – Shortest path problem

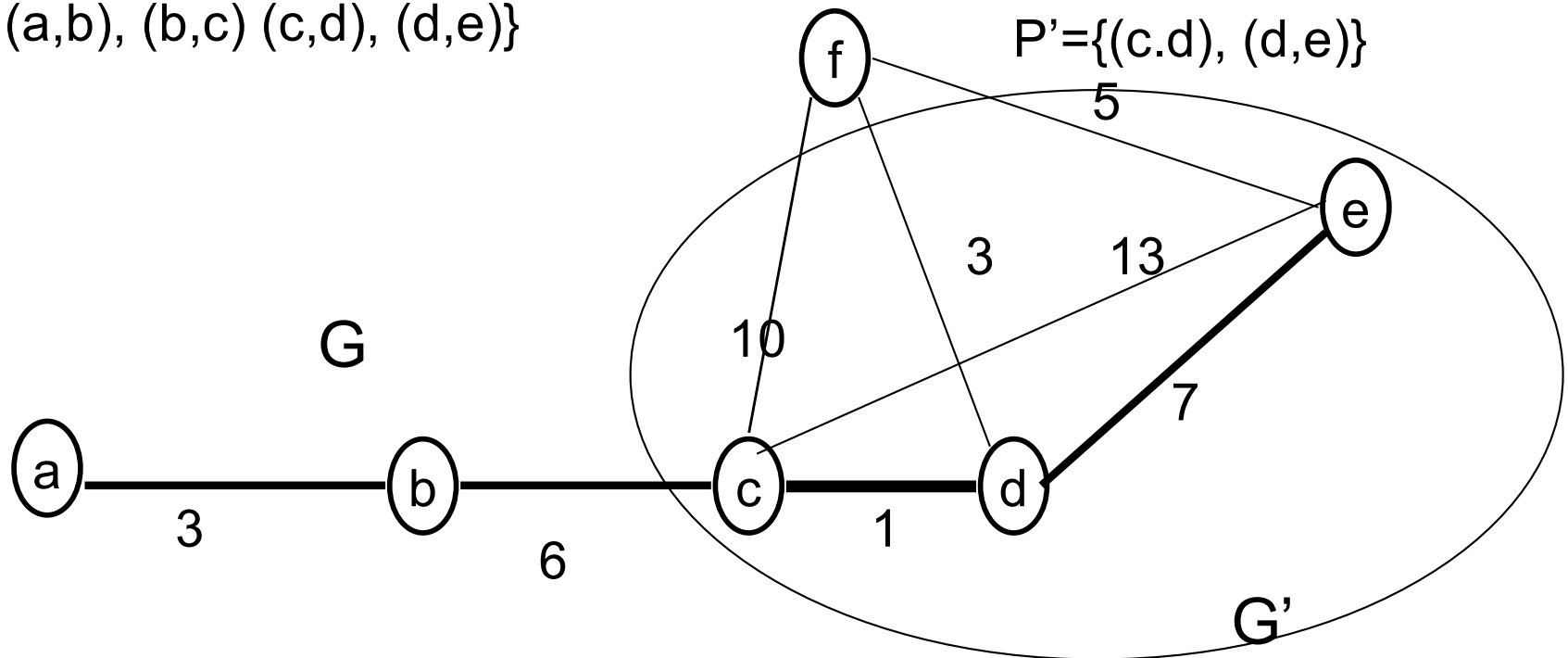
Problem: Given a graph G and vertices s and t , find a shortest path in G from s to t

Theorem: A subpath P' (from s' to t') of a shortest path P is a shortest path from s' to t' of the subgraph G' induced by P' .
Subpaths are paths that start or end at an intermediate vertex of P .

Proof: If P' was not a shortest path from s' to t' in G' , we can substitute the subpath from s' to t' in P by the shortest path in G' from s' to t' . The result is a shorter path from s to t than P . This contradicts our assumption that P is a shortest path from s to t .

Principle of optimality: Example

$P = \{(a,b), (b,c), (c,d), (d,e)\}$



P' must be a shortest path from c to e in G' . Otherwise, P cannot be a shortest path from a to e in G .

Principle of optimality

MST (minimum spanning tree) problem

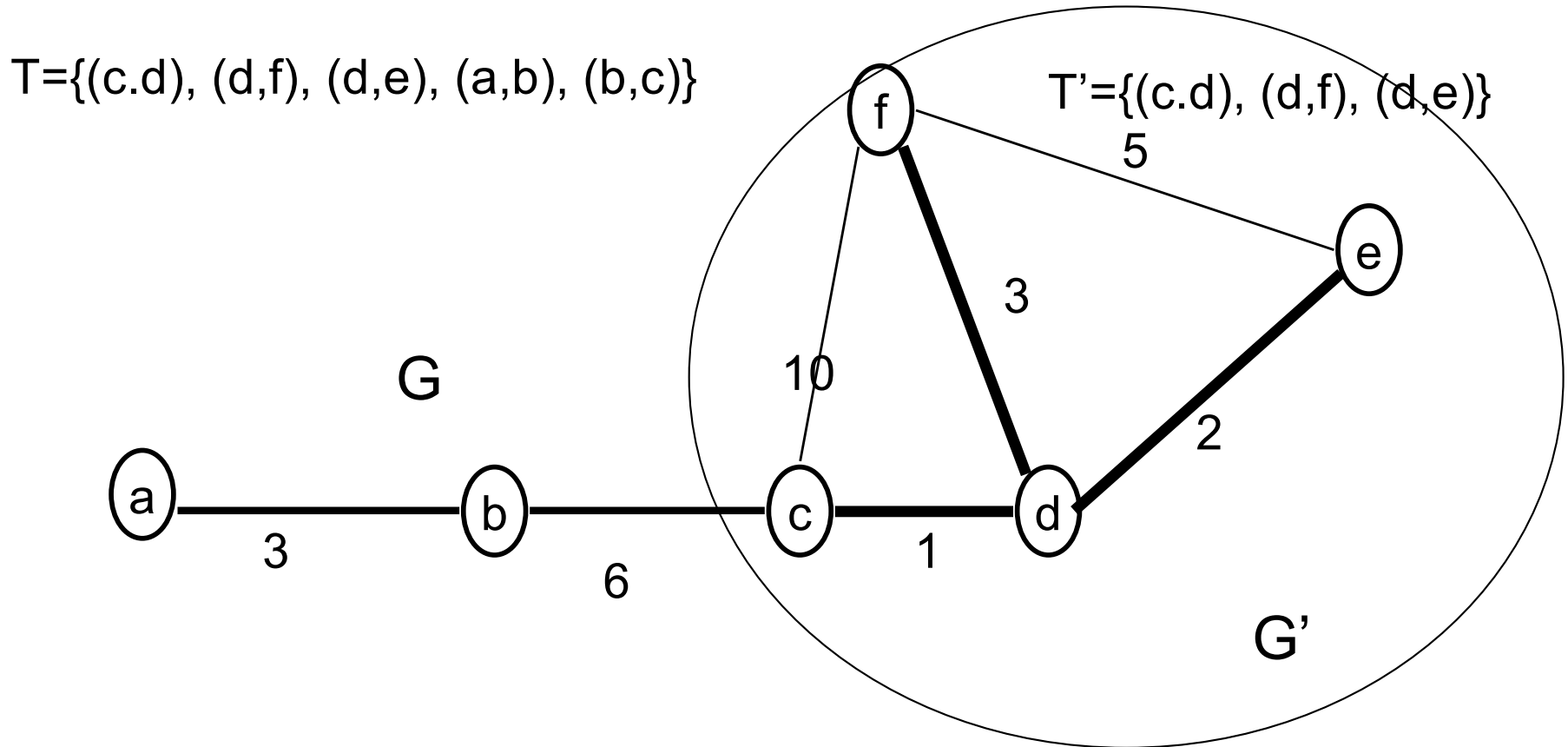
- Spanning tree: A tree connecting all the vertices in an undirected connected graph
- Minimum spanning tree: A spanning tree with the minimum weight

Principle of optimality for MST problem

- **Problem:** Given an undirected connected graph G , find a minimum spanning tree
- **Theorem:** Any subtree T' of an *MST* T of G is an *MST* of the subgraph G' of G induced by the vertices of the subtree.

Proof: If T' is not an *MST* of G' , we can substitute the edges of T' in T , with the edges of an *MST* of G' . This would result in a lower cost spanning tree, contradicting our assumption that T is an *MST* of G .

Principle of optimality

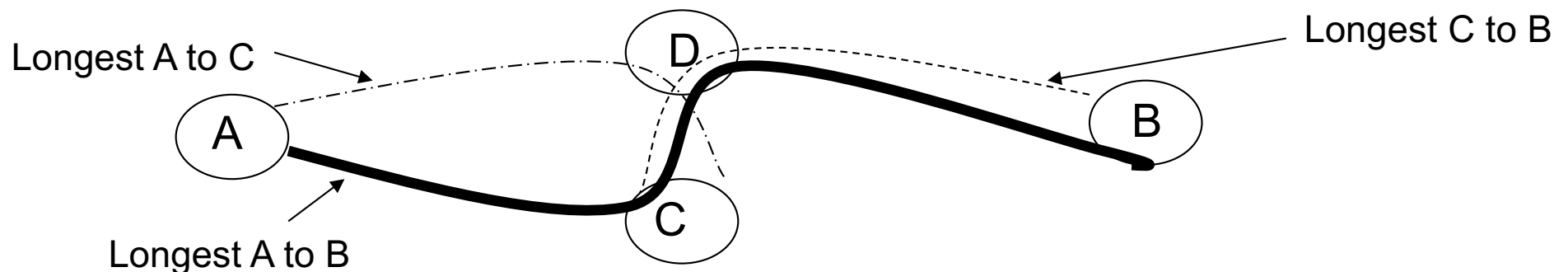


T' must be an MST of G' , otherwise T cannot be an MST

A problem that does not satisfy Principle of Optimality

Problem: What is the longest simple route between City A and B?

- Simple route $\rightarrow$ Never visit the same city twice.
- The longest simple route (solid line) has city C as an intermediate city.
- It does not consist of the longest simple route from A to C plus the longest simple route from C to B.



Floyd's Algorithm

All pairs shortest path

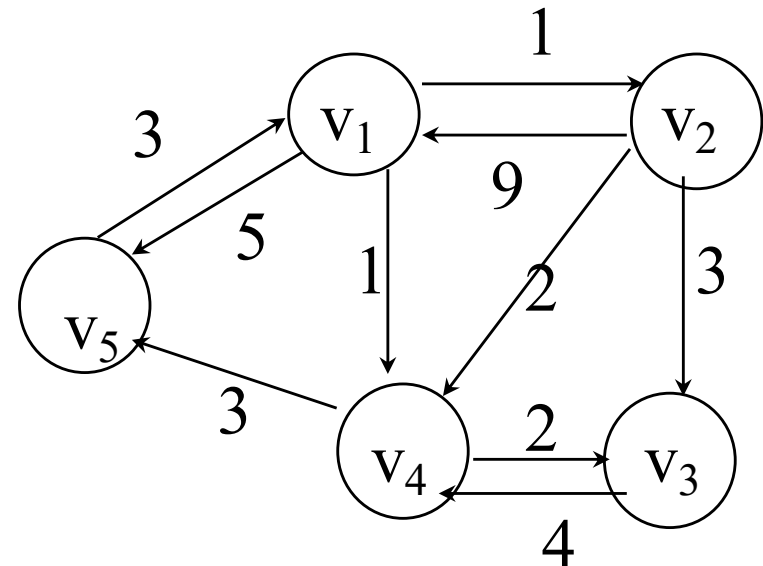
All pairs shortest path

- *The problem:* find the shortest path between every pair of vertices of a graph
 - Expensive using a brute-force approach
- *The graph* may contain no negative cycles
- *Representation:* a weight matrix where
 - $W(i,j)=0$ if $i=j$
 - $W(i,j)=\infty$ if there is no edge between i and j
 - $W(i,j)$ = “weight of edge”
- Note: we have shown principle of optimality applies to shortest path problems

The weight matrix and the graph

	1	2	3	4	5
1	0	1	∞	1	5
2	9	0	3	2	∞
3	∞	∞	0	4	∞
4	∞	∞	2	0	3
5	3	∞	∞	∞	0

Adjacency Matrix



The subproblems

- How can we define the shortest distance $d_{i,j}$ in terms of “smaller” problems?
- One way is to restrict the paths to only include vertices from a restricted subset.
- Initially, the subset is empty.
- Then, it is incrementally increased until it includes all the vertices.

The subproblems

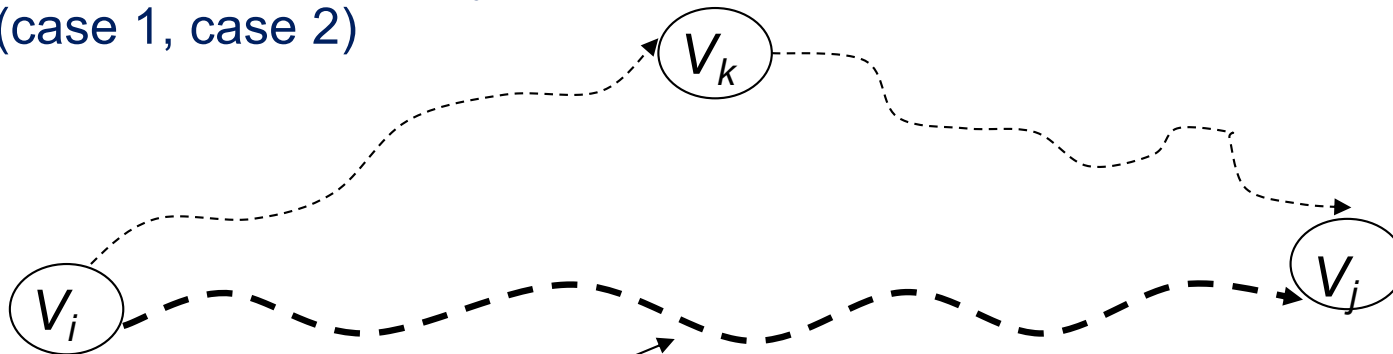
- Let $D^{(k)}[i,j]$ = weight of a shortest path from v_i to v_j using only vertices from $\{v_1, v_2, \dots, v_k\}$ as intermediate vertices in the path
 - $D^{(0)} = W$
 - $D^{(n)} = D$ which is the goal matrix
- How do we compute $D^{(k)}$ from $D^{(k-1)}$?

The Recursive Definition:

Case 1: A shortest path from v_i to v_j , which only uses vertices in $\{v_1, v_2, \dots, v_k\}$ as intermediate vertices, does not use $v_k \rightarrow$
 $D^{(k)}[i, j] = D^{(k-1)}[i, j]$

Case 2: A shortest path from v_i to v_j , which only uses vertices in $\{v_1, v_2, \dots, v_k\}$ as intermediate vertices, does use $v_k \rightarrow D^{(k)}[i, j] = D^{(k-1)}[i, k] + D^{(k-1)}[k, j]$

Shortest path from v_i to v_j using intermediate vertices $\{V_1, \dots, V_k\} = \min(\text{case 1, case 2})$



Shortest Path using intermediate vertices $\{V_1, \dots, V_{k-1}\}$

The recursive definition

- Since

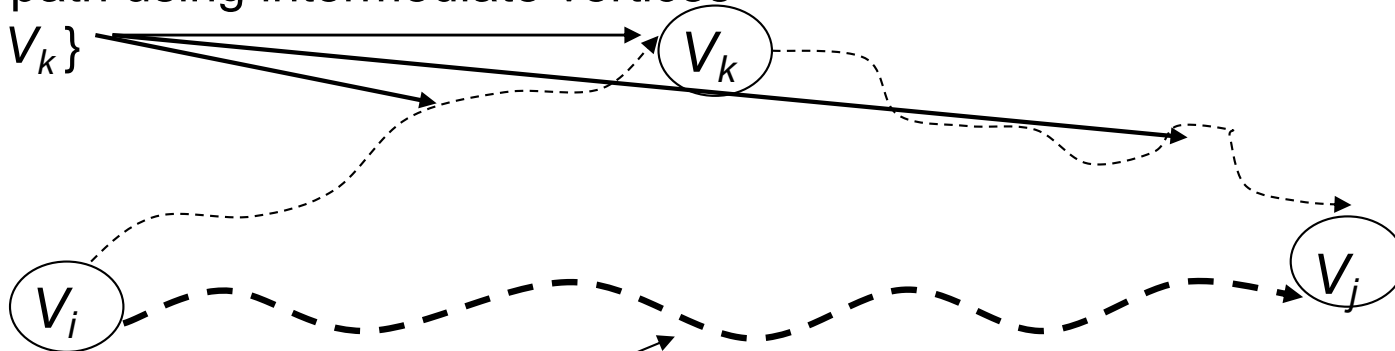
$$D^{(k)}[i,j] = D^{(k-1)}[i,j] \text{ or } D^{(k)}[i,j] = D^{(k-1)}[i,k] + D^{(k-1)}[k,j]$$

- We conclude:

$$D^{(k)}[i,j] = \min \{ D^{(k-1)}[i,j], D^{(k-1)}[i,k] + D^{(k-1)}[k,j] \}$$

Shortest path using intermediate vertices

$\{V_1, \dots, V_k\}$



Shortest Path using intermediate vertices $\{V_1, \dots, V_{k-1}\}$

The pointer array P

- Used to enable finding a shortest path
- Initially the array contains 0
- Each time a shorter path from i to j is found, save the k value that provided the minimum
- To print the intermediate nodes on the shortest path, execute a recursive procedure that prints the shortest paths from i and k , and from k to j

Floyd's Algorithm Using $n+1$ D matrices

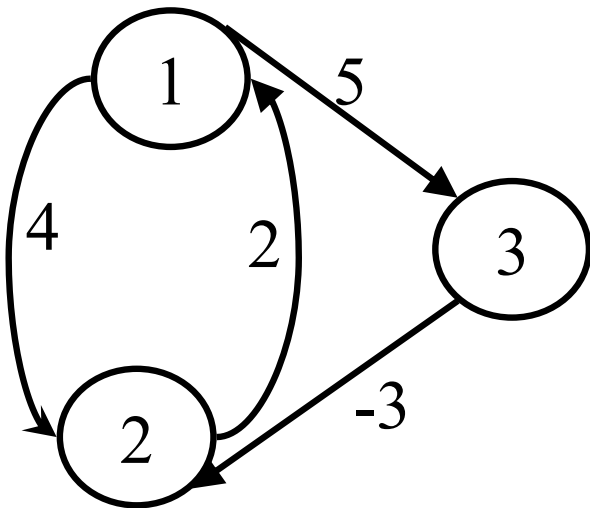
Floyd (input graph G)

// Compute shortest distance between all pairs of nodes in G ;

// Save P to find shortest paths

1. $D^0 \leftarrow W$ // initialize D array to W []
2. $P \leftarrow 0$ // initialize P array to [0]
3. for $k \leftarrow 1$ to n
4. for $i \leftarrow 1$ to n
5. for $j \leftarrow 1$ to n
6. if ($D^{k-1}[i, j] > D^{k-1}[i, k] + D^{k-1}[k, j]$)
7. then $D^k[i, j] \leftarrow D^{k-1}[i, k] + D^{k-1}[k, j]$
8. $P[i, j] \leftarrow k$;
9. else $D^k[i, j] \leftarrow D^{k-1}[i, j]$

Example



$$W = D^0 =$$

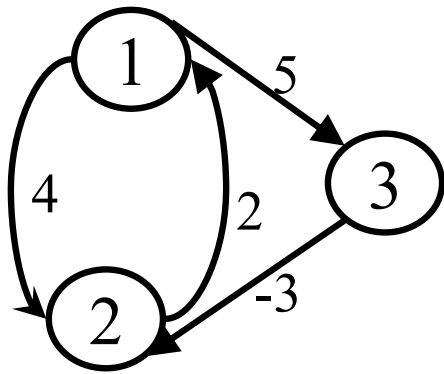
	1	2	3
1	0	4	5
2	2	0	∞
3	∞	-3	0

$$P =$$

	1	2	3
1	0	0	0
2	0	0	0
3	0	0	0

$k = 1$

Vertex 1 can be intermediate node



$D^1 =$

	1	2	3
1	0	4	5
2	2	0	7
3	∞	-3	0

$$\begin{aligned} D^1[2,3] &= \min(D^0[2,3], D^0[2,1] + D^0[1,3]) \\ &= \min(\infty, 7) \\ &= 7 \end{aligned}$$

$$\begin{aligned} D^1[3,2] &= \min(D^0[3,2], D^0[3,1] + D^0[1,2]) \\ &= \min(-3, \infty) = -3 \end{aligned}$$

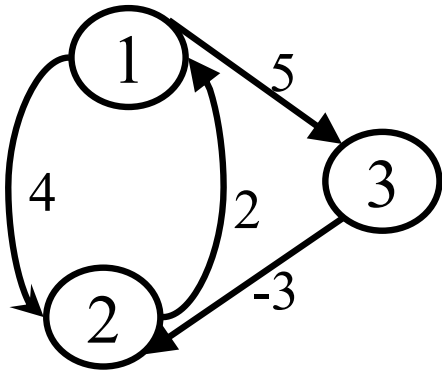
$D^0 =$

	1	2	3
1	0	4	5
2	2	0	∞
3	∞	-3	0

$P =$

	1	2	3
1	0	0	0
2	0	0	1
3	0	0	0

$k = 2$: Vertices 1, 2 can be intermediate



$$D^2 =$$

	1	2	3
1	0	4	5
2	2	0	7
3	-1	-3	0

$$P =$$

	1	2	3
1	0	0	0
2	0	0	1
3	2	0	0

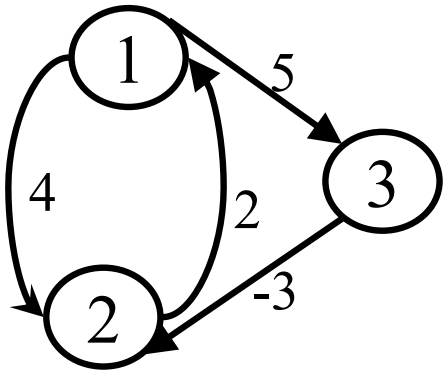
$$D^1 =$$

	1	2	3
1	0	4	5
2	2	0	7
3	∞	-3	0

$$\begin{aligned}
 D^2[1,3] &= \min(D^1[1,3], D^1[1,2] + D^1[2,3]) \\
 &= \min(5, 4 + 7) \\
 &= 5
 \end{aligned}$$

$$\begin{aligned}
 D^2[3,1] &= \min(D^1[3,1], D^1[3,2] + D^1[2,1]) \\
 &= \min(\infty, -3 + 2) \\
 &= -1
 \end{aligned}$$

$k = 3$: Vertices 1, 2, 3 can be intermediate



$$D^3 =$$

	1	2	3
1	0	2	5
2	2	0	7
3	-1	-3	0

$$P =$$

	1	2	3
1	0	3	0
2	0	0	1
3	2	0	0

$$D^2 =$$

	1	2	3
1	0	4	5
2	2	0	7
3	-1	-3	0

$$\begin{aligned}
 D^3[1,2] &= \min(D^2[1,2], D^2[1,3] + D^2[3,2]) \\
 &= \min(4, 5 + (-3)) \\
 &= 2
 \end{aligned}$$

$$\begin{aligned}
 D^3[2,1] &= \min(D^2[2,1], D^2[2,3] + D^2[3,1]) \\
 &= \min(2, 7 + (-1)) \\
 &= 2
 \end{aligned}$$

Floyd's Algorithm: Using two D matrices

Floyd

1. $D \leftarrow W$ // initialize D array to W []
2. $P \leftarrow 0$ // initialize P array to [0]
3. for $k \leftarrow 1$ to n
 // Computing D' from D
4. for $i \leftarrow 1$ to n
5. for $j \leftarrow 1$ to n
6. if ($D[i, j] > D[i, k] + D[k, j]$)
7. then $D'[i, j] \leftarrow D[i, k] + D[k, j]$
8. $P[i, j] \leftarrow k$;
9. else $D'[i, j] \leftarrow D[i, j]$
10. Move D' to D

Can we use only one D matrix?

- $D[i,j]$ depends only on elements in the k^{th} column and row of the distance matrix
- We will show that the k^{th} row and the k^{th} column of the distance matrix are unchanged when D^k is computed
- This means D can be calculated *in-place*

The main diagonal values

- Before we show that the k^{th} row and column of D remain unchanged, we show that the main diagonal remains 0
- $$\begin{aligned} D^{(k)}[j,j] &= \min\{ D^{(k-1)}[j,j], D^{(k-1)}[j,k] + D^{(k-1)}[k,j] \} \\ &= \min\{ 0, D^{(k-1)}[j,k] + D^{(k-1)}[k,j] \} \\ &= 0 \end{aligned}$$
- Based on which assumption?
 - There is no negative cycle

The k^{th} column

- k^{th} column of D^k is equal to the k^{th} column of D^{k-1}
- *Intuitively true* - a path from i to k will not become shorter by adding k to the allowed subset of intermediate vertices
- For all i ,
$$\begin{aligned} D^{(k)}[i,k] &= \min\{ D^{(k-1)}[i,k], D^{(k-1)}[i,k] + D^{(k-1)}[k,k] \} \\ &= \min\{ D^{(k-1)}[i,k], D^{(k-1)}[i,k] + 0 \} \\ &= D^{(k-1)}[i,k] \end{aligned}$$

The k^{th} row

- k^{th} row of D^k is equal to the k^{th} row of D^{k-1}
- *Intuitively true* - A path from k to j will not become shorter by adding k to the allowed subset of intermediate vertices
- For all j ,
$$\begin{aligned} D^{(k)}[k,j] &= \min\{ D^{(k-1)}[k,j], D^{(k-1)}[k,k] + D^{(k-1)}[k,j] \} \\ &= \min\{ D^{(k-1)}[k,j], 0 + D^{(k-1)}[k,j] \} \\ &= D^{(k-1)}[k,j] \end{aligned}$$

Floyd's Algorithm using a single D

Floyd

1. $D \leftarrow W$ // initialize D array to $W []$
2. $P \leftarrow 0$ // initialize P array to $[0]$
3. for $k \leftarrow 1$ to n
4. do for $i \leftarrow 1$ to n
5. do for $j \leftarrow 1$ to n
6. if ($D[i, j] > D[i, k] + D[k, j]$)
7. then $\{D[i, j] \leftarrow D[i, k] + D[k, j]$
8. $P[i, j] \leftarrow k; \}$

Time-complexity: $T(n) = n * n * n = n^3$

Printing intermediate nodes on shortest path from q to r

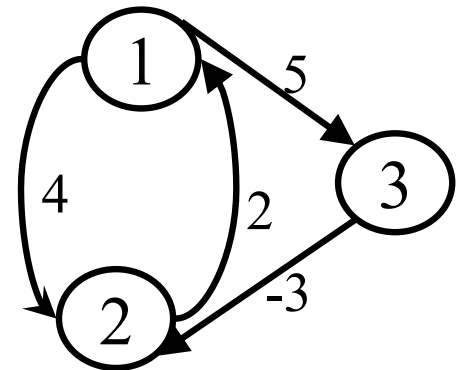
```
path(index q, r)
  if (P[ q, r ]!=0)
    path(q, P[q, r])
    println( "v"+ P[q, r])
    path(P[q, r], r)
  return;
```

```
//no intermediate nodes
else return
```

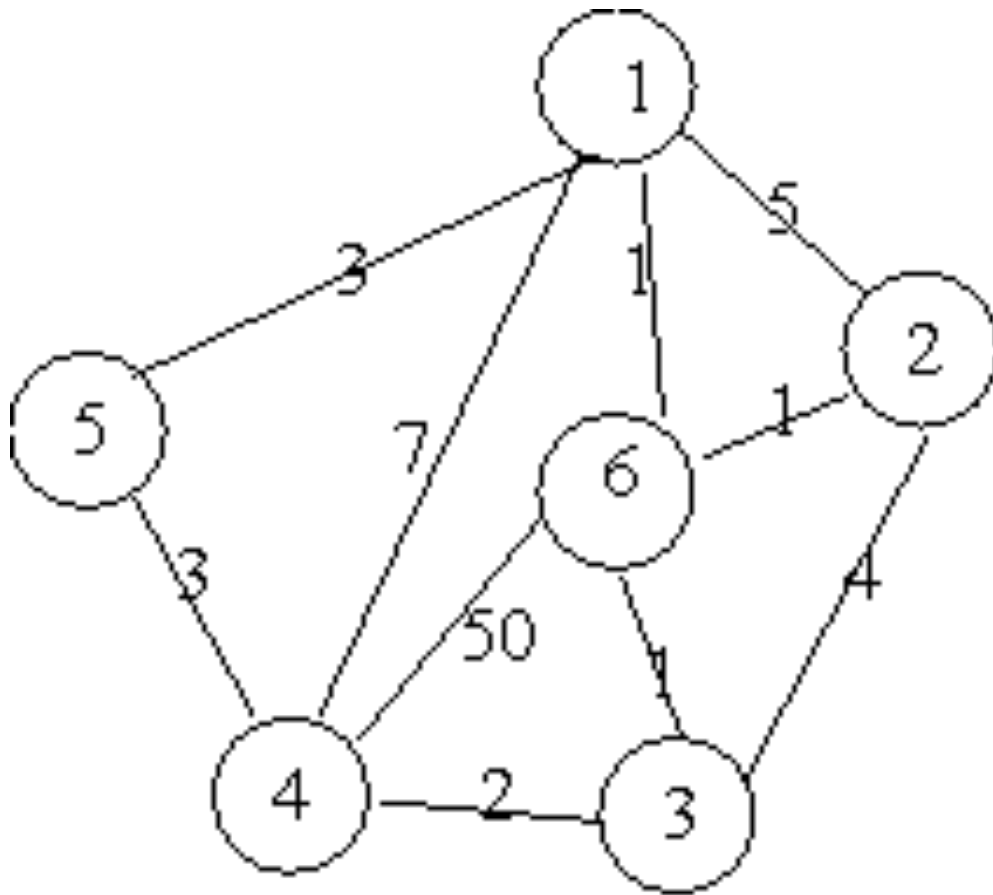
- If $D[q, r] < \infty$, print node q and call *path(...)*
- After returning from *path(...)*, print node r

P =

	1	2	3
1	0	3	0
2	0	0	1
3	2	0	0



Graph for a Floyd example

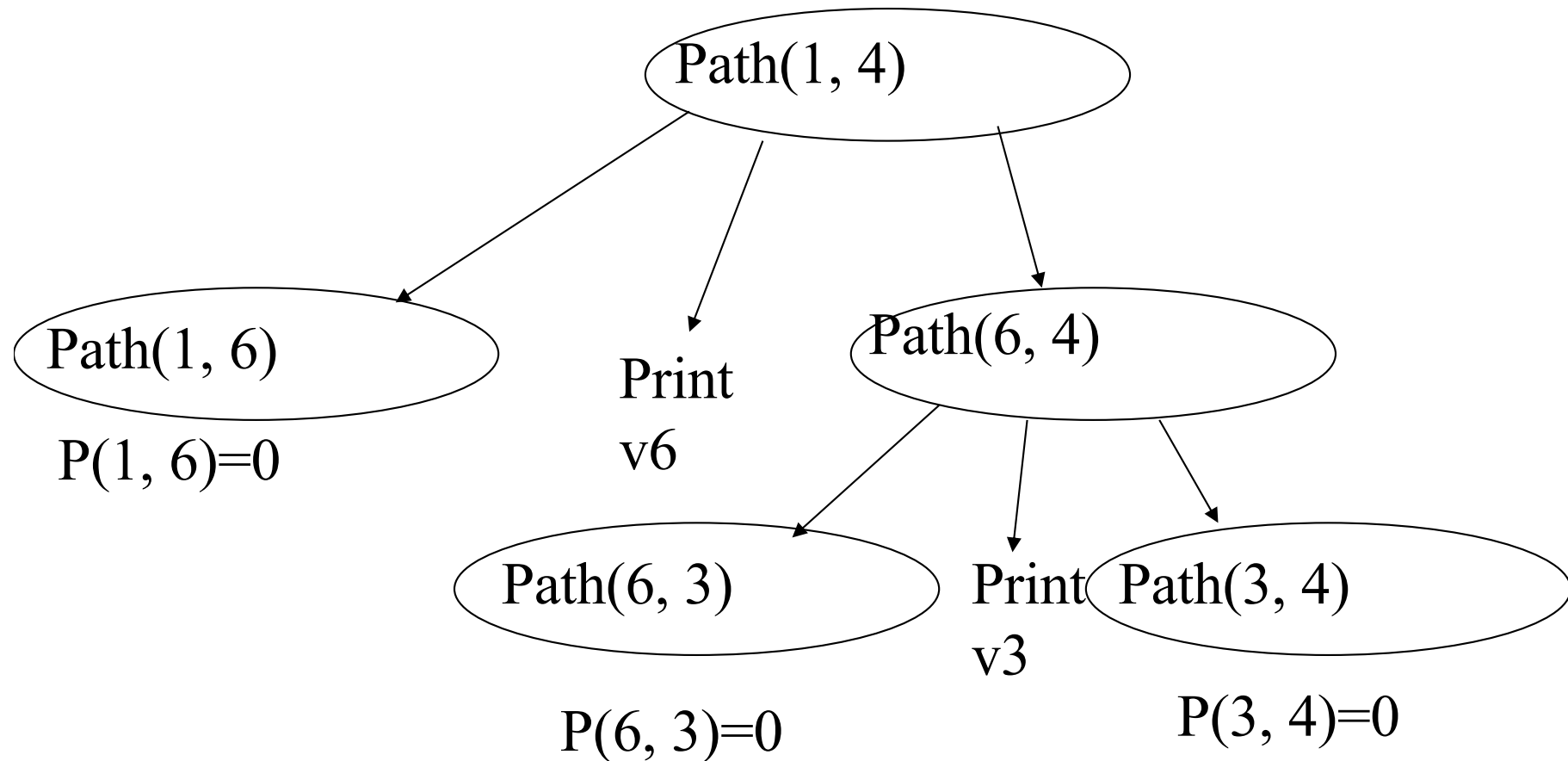


The final distance matrix D and P

	1	2	3	4	5	6
1	0	2(6)	2(6)	4(6)	3	1
2	2(6)	0	2(6)	4(6)	5(6)	1
$D^6 = 3$	2(6)	2(6)	0	2	5(4)	1
4	4(6)	4(6)	2	0	3	3(3)
5	3	5(6)	5(4)	3	0	4(1)
6	1	1	1	3(3)	4(1)	0

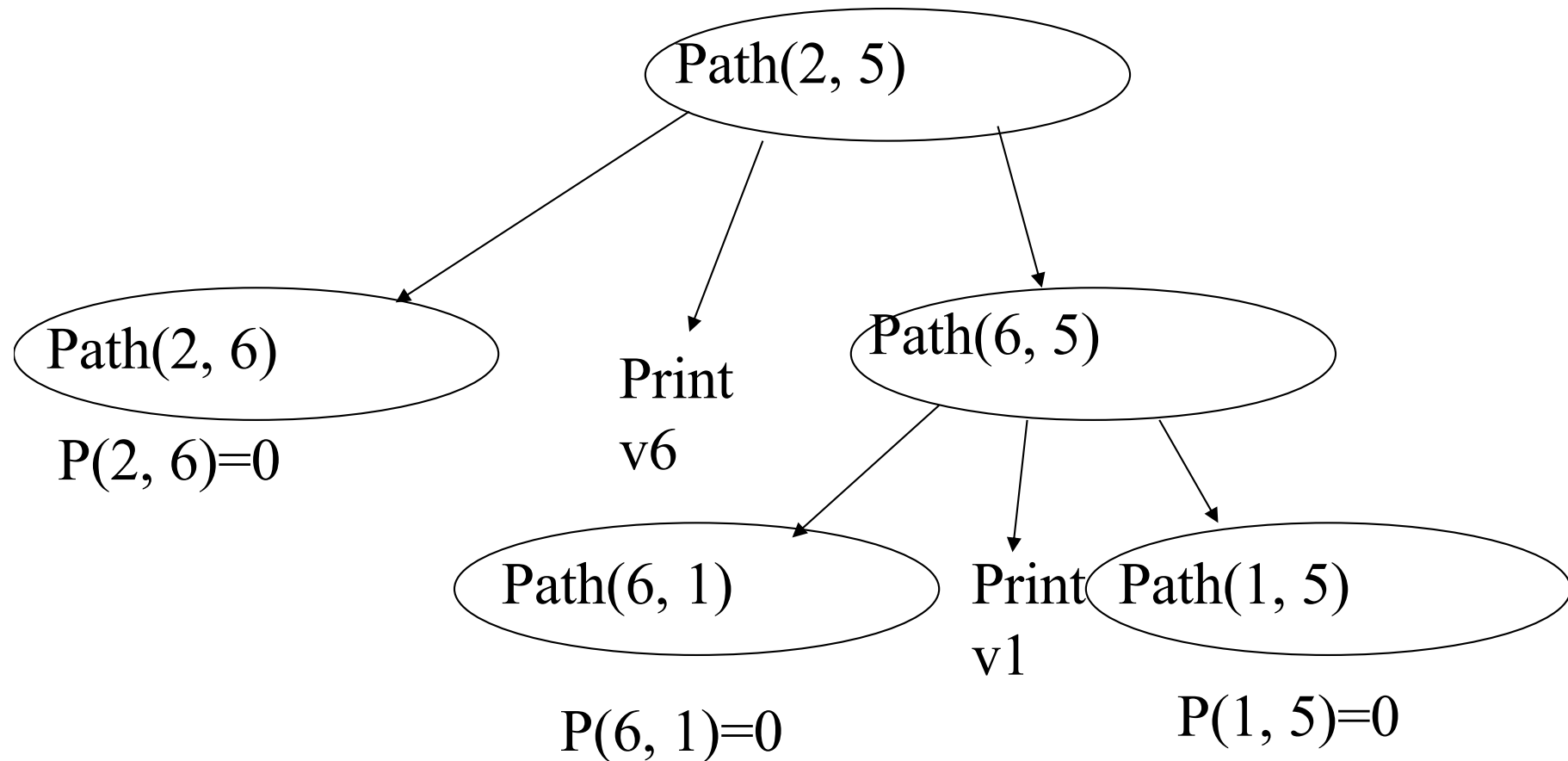
The values in parenthesis are the nonzero P values – intermediate nodes that minimize the distance.

The call tree for Path(1, 4)



The intermediate nodes on the shortest path from 1 to 4 are v6, v3.
The shortest path is v1, v6, v3, v4.

The call tree for Path(2, 5)



The intermediate nodes on the shortest path from 2 to 5 are v6, v1.
The shortest path is v2, v6, v1, v5.

Optimal Binary Search Tree

Binary Search Trees

- A binary tree of keys such that
 - Each node has one key
 - The keys in the left subtree of a given node are smaller than or equal to the item in the node
 - The keys in the right subtree of a given node are greater than the item in the node

Optimal Binary Search Tree

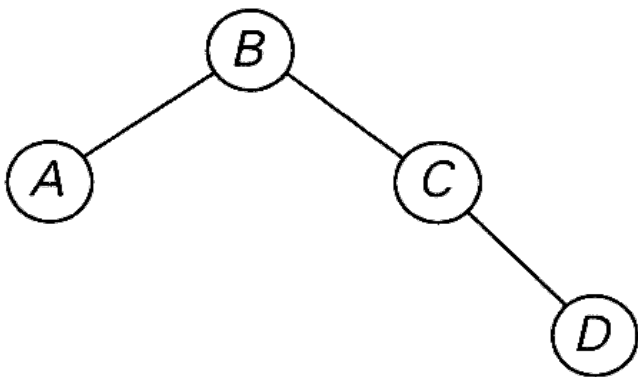
- Problem

- Given sequence $K = k_1 < k_2 < \dots < k_n$ of n sorted keys, with a search probability p_i for each key k_i .
- Build a binary search tree (BST) with minimum expected search cost.
- Actual cost = # of items examined.
- For key k_i , cost = $\text{depth}_T(k_i) + 1$, where $\text{depth}_T(k_i)$ = depth of k_i in BST T .

- Minimize the expected search time for a given probability distribution

Binary Search Trees

Index i	1	2	3	4
Key	A	B	C	D
$P(i)$	0.4	0.3	0.2	0.1



Average number of comparisons

$$\sum_{i=1}^n P(i) \cdot (\text{depth}_T(K_i) + 1)$$

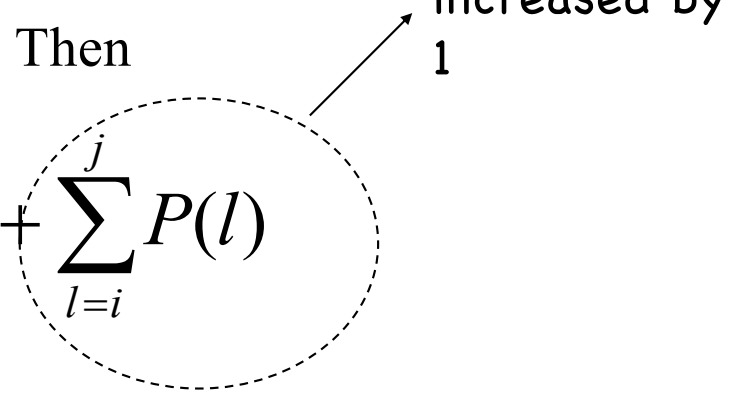
Optimal Binary Search Tree

Let T be a binary search tree that contains $K_i, K_{i+1}, \dots, K_j$ for some $1 \leq i \leq j \leq n$. We define the *cost of average search time* as

$$c(T) = \sum_{l=i}^j P(l) \cdot (\text{depth}_T(K_l) + 1)$$

Recurrence relation

Let T_L and T_R be the left and right sub-trees of T . Then

$$c(T) = c(T_L) + c(T_R) + \sum_{l=i}^j P(l)$$


Tree depth is increased by 1

Optimal Binary Search Tree

- *Principle of optimality*
- Let T be a binary search tree that has the minimum cost among all trees containing keys $K_i, K_{i+1}, \dots, K_j$
- Let K_m be the key at the root of T . So, $i \leq m \leq j$.
- T_L , the left subtree of T , is a binary search tree that has the minimum cost among all trees containing keys $K_i, \dots, K_{m-1}$
- T_R , the right subtree of T , is a binary search tree that has minimum cost among all trees containing keys $K_{m+1}, \dots, K_j$

Expected Search Cost

$E[\text{search cost in } T]$

$$= \sum_{i=1}^n (\text{depth}_T(k_i) + 1) \cdot p_i$$

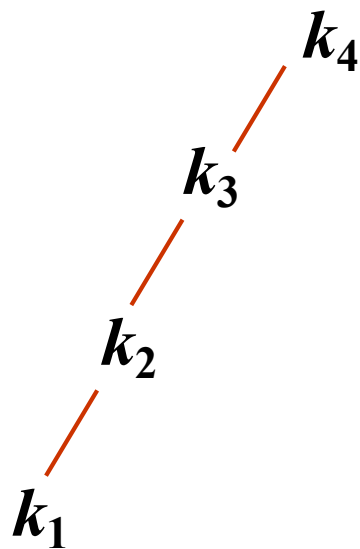
$$= \sum_{i=1}^n \text{depth}_T(k_i) \cdot p_i + \sum_{i=1}^n p_i$$

Sum of probabilities is 1.

$$= 1 + \sum_{i=1}^n \text{depth}_T(k_i) \cdot p_i$$

Example

- Consider 5 keys with these search probabilities:
 $p_1 = 0.1, p_2 = 0.2, p_3 = 0.3, p_4 = 0.4$

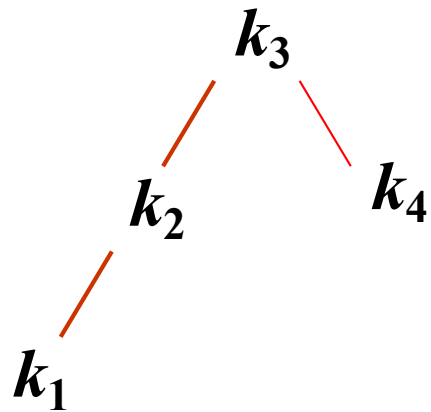


i	$\text{depth}_T(k_i)$	$\text{depth}_T(k_i) \cdot p_i$
1	3	0.3
2	2	0.4
3	1	0.3
4	0	0
Sum		1

Therefore, $E[\text{search cost}] = 2$

Example

- $p_1 = 0.1, p_2 = 0.2, p_3 = 0.3, p_4 = 0.4$



i	$\text{depth}_T(k_i)$	$\text{depth}_T(k_i) \cdot p_i$
1	2	0.2
2	1	0.2
3	0	0
4	1	0.4
		0.8

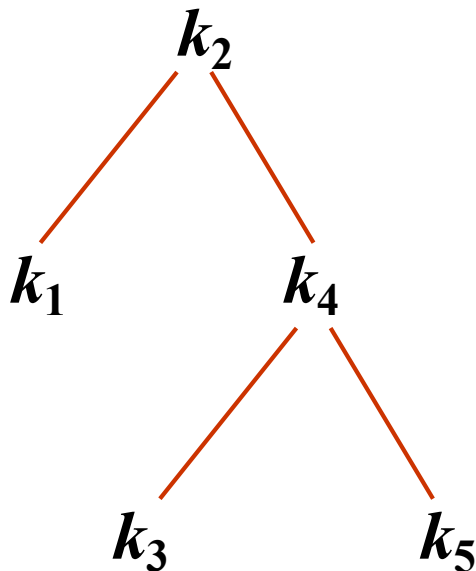
Therefore, $E[\text{search cost}] = 1.8$

This tree turns out to be optimal for this set of keys.

Optimal BST may not have the highest-probability key at root.

Example

- Consider 5 keys with these search probabilities:
 $p_1 = 0.25, p_2 = 0.2, p_3 = 0.05, p_4 = 0.2, p_5 = 0.3$.

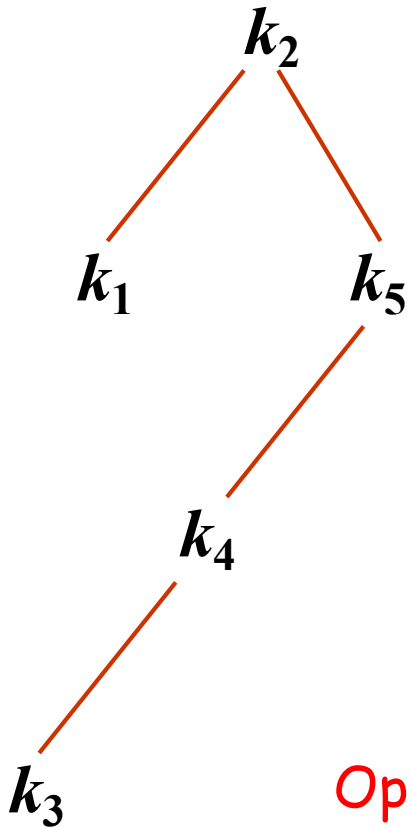


i	$\text{depth}_T(k_i)$	$\text{depth}_T(k_i) \cdot p_i$
1	1	0.25
2	0	0
3	2	0.1
4	1	0.2
5	2	0.6
		1.15

Therefore, $E[\text{search cost}] = 2.15$.

Example

- $p_1 = 0.25, p_2 = 0.2, p_3 = 0.05, p_4 = 0.2, p_5 = 0.3.$



i	$\text{depth}_T(k_i)$	$\text{depth}_T(k_i) \cdot p_i$
1	1	0.25
2	0	0
3	3	0.15
4	2	0.4
5	1	0.3
		1.10

Therefore, $E[\text{search cost}] = 2.10$.

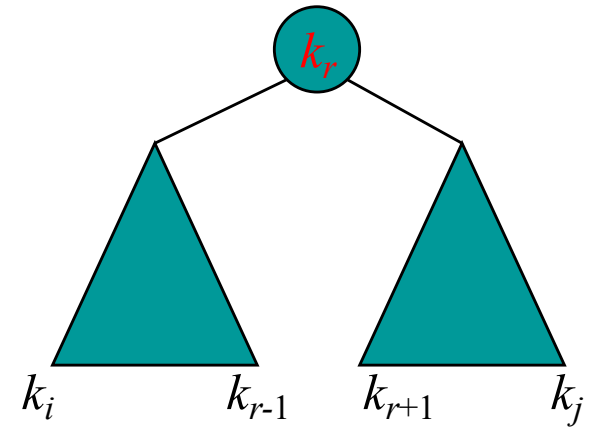
Optimal BST may not have the smallest height.

Example

- **Observations:**
 - Optimal BST **may not** have the smallest height.
 - Optimal BST **may not** have the highest-probability key at root.
- Build by exhaustive checking? (brute force)
 - Construct every possible n -node BST
 - For each BST, assign keys and compute expected search cost
 - Not efficient

Optimal Substructure

- One of the keys in $k_i, \dots, k_j$, say k_r , where $i \leq r \leq j$, must be the root of an optimal subtree for these keys.
- Left subtree of k_r contains $k_i, \dots, k_{r-1}$
- Right subtree of k_r contains $k_{r+1}, \dots, k_j$



- To find an optimal BST:
 - Examine all candidate roots k_r , for $i \leq r \leq j$
 - Determine all optimal BSTs containing $k_i, \dots, k_{r-1}$ and containing $k_{r+1}, \dots, k_j$
- Reuse optimal BSTs to find a bigger optimal BST with more nodes → Dynamic programming!

Computing an Optimal Solution

For each subproblem (i, j) , store:

- expected search cost in a table $A[1..n+1, 0..n]$
 - Will use only entries $A[i, j]$, where $j \geq i-1$.
- $\text{Root}[i, j]$ = root of subtree with keys $k_i, \dots, k_j$, for $1 \leq i \leq j \leq n$.
- $P[1..n+1, 0..n]$ = sum of probabilities
 - $p[i, i-1] = 0$ for $1 \leq i \leq n$.
 - $p[i, j] = p[i, j-1] + p_j$ for $1 \leq i \leq j \leq n$.
- *The Optimal Solution is:*
 - $A[1, n]$ (minimum expected search time with keys from 1 to n)
 - $\text{Root}[1, n] = R$ ($1 \leq r \leq n$)
 - Make an optimal binary search tree based on the root at R with keys $k_1, \dots, k_n$.

Average search time for the optimal tree

- The average search time for the tree k is given by:

$$\begin{aligned}
 & \underbrace{A[1][k-1]}_{\text{Average time in left subtree}} + \underbrace{p_1 + \dots + p_{k-1}}_{\text{Additional time comparing at root}} + \underbrace{p_k}_{\text{Average time searching for root}} + \underbrace{A[k+1][n]}_{\text{Average time in right subtree}} + \underbrace{p_{k+1} + \dots + p_n}_{\text{Additional time comparing at root}} \\
 &= A[1][k-1] + A[k+1][n] + \sum_{m=1}^n p_m
 \end{aligned}$$

For each key, there is one additional comparison at the root.

Average search time in this subtree is $A[1][k-1]$

Average search time in this subtree is $A[k+1][n]$

$Key_1, \dots, Key_{k-1}$

$Key_{k+1}, \dots, Key_n$

Key_k

$A[1][n] = \underset{1 \leq k \leq n}{\text{minimum}} (A[1][k-1] + A[k+1][n]) + \sum_{m=1}^n p_m$

1

- One of the k trees must be optimal, the average search time for the optimal tree is:

Generalize the Average search time for the optimal tree

- The average search time for the tree with keys from Key_i to key_j:

$$A[i][j] = \underset{i \leq k \leq j}{\text{minimum}}(A[i][k-1] + A[k+1][j]) + \sum_{m=i}^j p_m \quad i < j$$

$$A[i][i] = p_i$$

$A[i][i-1]$ and $A[j+1][j]$ are defined to be 0.

- $A[i][j]$ is computed from entries in the i th row but to the left of $A[i][j]$ and from entries in the j th column but beneath $A[i][j]$.
- We proceed by computing in sequence the values on each diagonal.

Optimal binary search tree – Algorithm

```
Void optsearchtree(int n, const float p[], float& minavg, index R[][])  
{  
  integer i, j, k, diagonal;  
  float A[1..n+1][0..n];  
  for (i = 1; i <= n; i++) //initialization  
  {  
    A[i][i-1] = 0;  
    R[i][i-1] = 0;  
    A[i][i] = p[i];  
    R[i][i] = i;  
  }  
  
  A[n+1][n] = 0;  
  R[n+1][n] = 0;
```

Optimal binary search tree – Algorithm (Cont.)

```
for (diagonal = 1; diagonal <= n-1; diagonal++)  
    for (i = 1; i <= n - diagonal; i++)  
    {  
        j = i + diagonal;
```

$$A[i][j] = \min_{i \leq k \leq j} (A[i][k-1] + A[k+1][j]) + \sum_{m=i}^j p_m$$

```
        R[i][j] = a value of k that gave the minimum  
    }
```

```
minavg = A[1][n];  
}
```

Time Complexity: $O(n^3)$

- The nested loop iterates $O(n^2)$ and each iteration takes $O(n)$ to find the minimum

Optimal binary search tree – Algorithm (Cont.)

- Basic operation: The instructions executed for each value of k . They include a comparison to test for the minimum.
- Input size: n , the number of keys.
- We have a loop within a loop within a loop. Because $j = i + diagonal$, for given values of $diagonal$ and i , the number of passes through the k loop is
 - $j-i+1 = i+diagonal-i+1 = diagonal + 1$
- For a given value of $diagonal$, the number of passes through the **for- i** loop is $n - diagonal$. Because $diagonal$ goes from 1 to $n - 1$, the total number of times the basic operation is done equals

$$\sum_{diagonal=1}^{n-1} [(n - diagonal) \times (diagonal + 1)]$$

- This expression is equal to $\frac{n(n-1)(n+1)}{6} \in \Theta(n^3)$ (for your exercise).

Build Optimal Binary Search tree

Problem: Build an optimal binary search tree.

Inputs: n , the number of keys, an array Key containing the n keys in order, and the array R produced by optsearchtree Algorithm.

$R[i][j]$ is the index of the key in the root of an optimal tree containing the i th through the j th keys.

Outputs: a pointer $tree$ to an optimal binary search tree containing the n keys.

```
node_pointer tree (index i, j)
{
    index k;
    node_pointer p;

    k = R[i][j];
    if (k == 0)
        return NULL;
    else {
        p = new nodetype;
        p->key = Key[k];
        p->left = tree(i, k - 1);
        p->right = tree(k + 1, j);
        return p;
    }
}
```


Longest Common Subsequence (LCS)

Longest Common Subsequence (LCS)

- Problem: Given sequences $x[1..m]$ and $y[1..n]$, find a **longest common subsequence** of both.
- Example: $x=ABCBDAB$ and $y=BDCABA$
 - BCA is a common subsequence and
 - BCBA and BDAB are two LCSs
 - Note: A substring is contiguous, but a subsequence doesn't have to be.
- Applications
 - Pattern Analysis
 - Bioinformatics

e.g. two DNA sequences:
-AACAGTTACC
TAA- GGT - -CA

LCS

- Come up with a brute force solution and evaluate it
- Write a recurrence equation
- Dynamic programming solution

Brute force solution

- **Solution:** For every subsequence of x , check if it is a subsequence of y .
- **Analysis :**
 - Each subsequence of X corresponds to a subset of the indices $\{1, 2, \dots, m\}$ of X . There are 2^m subsequences of x .
$$C_m^1 + C_m^2 + C_m^3 + \dots + C_m^m = 2^m$$
 - Each check takes $O(n)$ time, since we scan y for the first element, and then scan for the second element, etc.
 - The worst case running time is $O(n2^m)$.

Writing the recurrence equation

- Let X_i denote the *i^{th} prefix*, i.e., $x[1..i]$, of $x[1..m]$, and X_0 denotes an empty prefix
- We will first compute the *length of an LCS of X_m and Y_n* , $\text{LenLCS}(m, n)$, and then use the length information to find the actual subsequence
- We need a recursive formula for computing $\text{LenLCS}(i, j)$

Writing the recurrence equation

- If X_i and Y_j end with the same character $x_i=y_j$, the LCS must include the character. If it did not, we could get a longer LCS by adding the common character.
- If X_i and Y_j do not end with the same character there are two possibilities:
 - Either the LCS does not end with x_i ,
 - or it does not end with y_j
- Let Z_k denote an LCS of X_i and Y_j

X_i and Y_j end with $x_i = y_j$

X_i

x_1 x_2 $\dots$ x_{i-1}	x_i
-------------------------------	-------

Y_j

y_1 y_2 $\dots$ y_{j-1}	$y_j = x_i$
-------------------------------	-------------

Z_k

z_1 $z_2 \dots z_{k-1}$	$z_k = y_j = x_i$
---------------------------	-------------------

Z_k is Z_{k-1} followed by $z_k = y_j = x_i$ where
 Z_{k-1} is an LCS of X_{i-1} and Y_{j-1} and
 $LenLCS(i, j) = LenLCS(i-1, j-1) + 1$

X_i and Y_j end with $x_i \neq y_j$

X_i

x_1	x_2	$\dots$	x_{i-1}	x_i
-------	-------	---------	-----------	-------

X_i

x_1	x_2	$\dots$	x_{i-1}	x_i
-------	-------	---------	-----------	-------

Y_j

y_1	y_2	$\dots$	y_{j-1}	y_j
-------	-------	---------	-----------	-------

Y_j

y_1	y_2	$\dots$	y_{j-1}	y_j
-------	-------	---------	-----------	-------

Z_k

z_1	z_2	$\dots$	z_{k-1}	$z_k \neq y_j$
-------	-------	---------	-----------	----------------

Z_k

z_1	z_2	$\dots$	z_{k-1}	$z_k \neq x_i$
-------	-------	---------	-----------	----------------

Z_k is an LCS of X_i and Y_{j-1}

Z_k is an LCS of X_{i-1} and Y_j

$$\text{LenLCS}(i, j) = \max\{\text{LenLCS}(i, j-1), \text{LenLCS}(i-1, j)\}$$

The recurrence equation

$$\text{lenLCS}(i, j) = \begin{cases} 0 & \text{if } i = 0, \text{ or } j = 0 \\ \text{lenLCS}(i - 1, j - 1) + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j \\ \max \{ \text{lenLCS}(i - 1, j), \text{lenLCS}(i, j - 1) \} & \text{otherwise} \end{cases}$$

Dynamic programming solution

- Initialize the first row and the first column of the matrix $LenLCS$ to 0
- Calculate $LenLCS(1, j)$ for $j = 1, \dots, n$
- Then the $LenLCS(2, j)$ for $j = 1, \dots, n$, etc.
- Store also in a table an *arrow* pointing to the array element that was used in the computation.
- Hence, the computation is $O(mn)$

Example

	y_j	B	D	C	A
x_i	0	0	0	0	0
A	0	↑ 0	↑ 0	↑ 0	↖ 1
B	0	↖ 1	← 1	← 1	↑ 1
C	0	↑ 1	↑ 1	↖ 2	← 2
B	0	↖ 1	↑ 1	↑ 2	↑ 2

To find an LCS follow the arrows, for each diagonal arrow there is a member of the LCS

LCS-Length (X, Y)

```
m ← length[X]
n ← length[Y]
for i ← 1 to m do
    c[i, 0] ← 0
for j ← 1 to n do
    c[0, j] ← 0
```

LCS-Length (X, Y) cont.

```
for i ← 1 to m do
  for j ← 1 to n do
    if  $x_i = y_j$ 
       $c[i, j] \leftarrow c[i-1, j-1] + 1$ 
       $b[i, j] \leftarrow \text{"D"}$ 
    else
      if  $c[i-1, j] \geq c[i, j-1]$ 
         $c[i, j] \leftarrow c[i-1, j]$ 
         $b[i, j] \leftarrow \text{"U"}$ 
      else
         $c[i, j] \leftarrow c[i, j-1]$ 
         $b[i, j] \leftarrow \text{"L"}$ 
  return c and b
```

Memoization

Idea

- Memoization is a **top-down** approach for dynamic programming. It is used for recursive code that solves sub problems more than once.
- The idea is to modify the recursive code, and use a table to store the solution for every problem that was already solved.
- Logically equivalent to recursive calls, but recursive calls are replaced with table look-ups for more efficiency

Initialization

- Before applying the recursive code, you need to initialize the table to some "impossible value".
- For example, if the function can have only positive values you can initialize the table to negative values.

Method

- The recursive code is changed to first check the table to determine whether you have already solved the current problem or not.
- If the value in the table is still the initial value, you know that you have not yet solved the current problem. So, you solve the subproblem and store the solution in the table.
- Otherwise, you know that the value is the required solution and use or return it directly as appropriate.

Example 1

The following code is for a memoized version of recursive Fibonacci.

- MemoizedFib(n)
 - for (i=0; i<=n; i++)
 - A[i]=-1 //initialize the array
 - return LookupFib(n)
 -
 -
 - LookupFib(n) //compute fn if not yet done
 - **if** A[n]== -1 //initial value
 - **if** n<=1 //base case
 - A[n] = n //store solution
 -
 - //solve and store a general case
 - **else** A[n] = (LookupFib(n-1)+LookupFib(n-2))
 -
 - **return** A[n] //return $f_n = A[n]$
- If $A[n] \neq -1$, simply return $A[n]$ making no more recursive calls!!!

Example 2

The following code is for a memoized version of the recursive code for binomial coefficients

- `memoizedBC(n, k)`
 - `for i=0 to n`
 - `for j=0 to min(i, k)`
 - `B[i, j] = -1 //initialize array`
 - `return binomialCoef(n, k)`
 - `binomialCoef(n, k)`
 - `if B[n, k] == -1 //B[n, k] not yet computed`
 - `if k = 0 or k = n //Base case`
 - `B[n, k] = 1 //solve base case and store`
 - `//solve and store a general case`
 - `else B[n, k] = (binomialCoef(n-1, k-1) + binomialCoef(n-1, k))`
 - `return B[n, k]`
- If $B[n, k] \neq -1$, simply return $B[n, k]$ with no recursive calls!!!

Summary

- Dynamic programming is a powerful tool
- Bottom up approach
- Store results for smaller instances and reuse them to solve a bigger instance
- Now let's move on to graphs and greedy algorithms